SYSTEM DESIGN

GETTING STARTED WITH SYSTEM DESIGN

SYSTEM DESIGN:

System design is the process of designing the architecture and components of a software system to meet specific business requirements. Involves translating user requirements into a detailed blueprint that guides the implementation phase. The goal is to create a well-organized and efficient structure that meets the intended purpose while considering factors like scalability, maintainability, and performance.

System design in SDLC:

In SDLC (Software development life cycle), without the designing phase, one can't jump to the implementation or testing part. System Design is a vital step and also provides the backbone to handle exceptional scenarios because it represents the **business logic** of the software.

SDLC: Analyze > Plan > Maintain > Implement > Develop

Divided into two complementary parts:

Low level design:

- Detailed class & module design
- Defines logic, data structures, APIs
- Focus on implementation & maintainability
- Useful for developers & engineers

High level design:

- Big picture architecture
- Defines system components & interactions
- Focus on scalability, reliability, performance
- Useful for architects and stakeholders

High-level design:

It lays out the overall architecture of the system – how major components interact, what services or modules will exist, and how data flows among them. Gives you the big picture: how the system fits together, its core structure, and major decisions. Done by architects, stakeholders and managers

Topics covered in HLD:

- **System architecture overview**: Defines the major components, modules, and how they interact (e.g., services, queues, databases)
- **Data flow and component interaction**: Illustrates how data moves between modules, along with key integrations and interfaces
- **Technology stack and infrastructure**: High-level decisions on frameworks, platforms, hardware, databases, and hosting setups
- **Module responsibilities**: Describes what each module does and how they relate to one another
- **Performance & trade-offs**: Includes design trade-offs, performance considerations, scalability, security, cost, and other non-functional factors
- **Artifacts**: Commonly includes architecture diagrams, component and deployment diagrams, data flow diagrams, and possibly ER/DB schematic overviews.

Low-level design:

It covers how each part works & is implemented internally. Gives developers a clear, actionable blueprint to build each component. Once a High Level Design is built by stakeholders, it is the job of senior developers and designers to create a low level design.

Topics covered in LLD:

- **Component/module breakdown**: Detailed internal logic for each module—with class responsibilities, methods, attributes, interactions
- **Database schema & structure**: Designing tables, keys, indexes, relationships with SQL/NoSQL refinements
- **API & interface definitions**: Precise request/response formats, error codes, methods, endpoints, and internal interfacing
- **Error handling & validation logic**: Define how each module manages invalid inputs, failures, edge cases, and logging
- **Design patterns & SOLID**: Implement design patterns and solid principles to ensure clean, extensible, maintainable code

- **UML and pseudocode artifacts**: Class diagrams, sequence diagrams, pseudocode or flowcharts to clarify logic paths and method calls

Steps for getting started with system design:

Understand the requirements

Define the system architecture

Choose the technology stack

Design the modules

Plan for scalability

Consider security and privacy

Test and validate

IMPORTANCE OF SYSTEM DESIGN

In today's world, applications must be able to handle millions of users at time, process huge amounts of data in real time, and remain available 24/7. From social media platforms to financial systems, the demand for **scalable, reliable, and efficient** software has never been higher. This is where **System Design** comes in and provides the blueprint for building robust systems that meet these challenges head-on.

Without a well-designed system architecture, applications frequently fail, experience poor performance results high maintenance costs. The following points highlight the critical need for a robust system design architecture.

- To Build Scalable Systems
- To Ensure Reliability
- To Optimize Performance
- To Reduce Costs
- To Prepare for the Unexpected
- To Collaborate Effectively
- To Future-Proof Systems

SYSTEM DESIGN VS. SOFTWARE DESIGN

System design:

Process of defining the architecture, components, modules, interfaces, and data for a system to satisfy specified requirements. It involves translating user requirements into a detailed blueprint that guides the implementation phase. The goal is to create a well-organized and efficient structure that meets the intended purpose while considering factors like scalability, maintainability, and performance.

Software design:

Software Design is the process of defining the architecture, structure, behavior, and implementation details of software systems to meet specific requirements effectively and efficiently. It involves translating user needs and functional specifications into a blueprint that guides the development process.

Components of System Design:

Involves integrating hardware, software, network, data, and processes to create a comprehensive solution that addresses specific requirements.

Hardware components – physical devices such as servers, computers, storage devices, networking equipment, sensors, and other peripherals

Software components – programs, applications, and operating systems

Network infrastructure – network architecture, protocols, and communication channels that enable data transfer and interaction between system components, both within the local environment and across distributed networks.

Data management – Involves designing databases, data models, data storage and retrieval mechanisms

Processes and workflows – Includes defining the sequence of tasks, activities and operations to accomplish specific goals or functions within the system

Security mechanisms – Access control, encryption, authentication and authorization

Scalability and performance considerations – Designing the system architecture and infrastructure to accommodate increasing loads, user bases, and data volumes

Reliability and fault tolerance – Implementing redundancy, failover mechanisms, and error handling strategies

User interface and user experience – Focus on designing intuitive interfaces, workflows, and interactions to enhance user satisfaction, productivity, and accessibility

Integration and interoperability – Involves ensuring seamless interaction and compatibility between different system components, external systems, and third-party services.

Components of software design:

Encompasses various components that contribute to the creation of a well-structured, maintainable, and efficient software system.

Architecture design – Involves defining the overall structure and organization of the software system

Module design – Involves breaking down the system into smaller, cohesive units or modules

Data design – Focuses on designing the data structures, databases, and data models

UI – Involves creating intuitive and user-friendly interfaces

Algorithm design – Determining how tasks are performed efficiently

Eror handling and exception design – Focuses on designing mechanisms to detect, report and handle errors

Security design – Involves incorporating measures to protect the software system against unauthorized access, data breaches and malicious attacks

Performance design – Aims to optimize the software system's speed, responsiveness and resource utilization

Concurrency and multithreading design – Includes designing concurrent algorithms, synchronization mechanisms, and thread-safe data structures to ensure correctness and avoid race conditions.

Challenges in system design:

- **Scalability:** Designing a system that can handle increasing loads, data volumes, and user bases without compromising performance or functionality.
- **Reliability:** Ensuring continuous and dependable operation of the system, even in the face of hardware failures, software errors, or external disruptions.
- **Availability:** Designing redundancy and failover mechanisms to minimize downtime and ensure that critical services are always accessible to users.
- Performance Optimization: Balancing resource utilization, response times, and throughput to achieve optimal system performance under varying workloads and conditions.
- **Complex Systems Integration:** Integrating diverse hardware and software components, legacy systems, and third-party services while maintaining compatibility and interoperability.

- **Security:** Implementing robust security measures to protect sensitive data, prevent unauthorized access, and mitigate cybersecurity threats such as hacking, malware, and data breaches.
- **Legacy Systems:** Upgrading or migrating legacy systems to modern architectures while minimizing disruption and ensuring compatibility with existing infrastructure and processes.
- **Evolving Requirements:** Adapting the system design to accommodate changing business needs, technological advancements, and regulatory requirements over time.

Challenges in software design:

- **Requirements Management:** Gathering, understanding, and managing requirements from stakeholders can be complex. Ensuring that requirements are clear, complete, and consistent is essential to successful software design.
- **Scalability:** Designing software to handle increasing amounts of data, users, or transactions can be challenging. Ensuring that the architecture and design can scale efficiently without sacrificing performance is crucial.
- **Maintainability:** Writing code that is easy to understand, modify, and extend is essential for long-term success. Poorly designed software can become difficult and costly to maintain over time.
- **Flexibility and Extensibility:** Designing software that can adapt to changing requirements and environments is vital. Building a flexible and extensible architecture allows for easier integration of new features and technologies.
- **Performance Optimization:** Balancing performance requirements with other design considerations can be challenging. Optimizing algorithms, data structures, and system architecture to meet performance goals without sacrificing other qualities such as maintainability and scalability is crucial.
- **Security:** Ensuring that software is secure against various threats, such as unauthorized access, data breaches, and malicious attacks, is essential. Designing robust security measures into the architecture and implementing secure coding practices are critical aspects of software design.
- **Integration and Interoperability:** Integrating software components with external systems, APIs, and third-party services can be complex. Ensuring compatibility, reliability, and seamless interaction between different components is essential for interoperability.

SYSTEM DESIGN LIFE CYCLE

Defined as the complete journey of a System from planning to deployment.

7 Phases or stages:

Planning stage

Feasibility study stage

System design stage

Implementation stage

Testing stage

Deployment stage

Maintenance and support

Involves planning, creating, testing, and implementing a system. Includes defining system requirements, specifying how components will interact, and detailing the architecture.

Planning:

- This is where the project starts! You decide what you want to achieve (goals), how much you can spend (budget), and who will work on it (team).
- **Example:** A company plans to build a customer management system and outlines its key features and the team involved.

Feasibility study:

- Before starting, check if the idea is practical. Will it work technically? Is it affordable?
- **Example:** The company checks if building the system is worth the cost and effort and whether it can deliver the expected benefits.

System design:

- This stage is all about planning how the system will work behind the scenes and how it will look to users.
- Imagine designing a house, before construction begins, architects create blueprints that show where every room, pipe, and wire will go. Similarly, in system design, you create a detailed plan that developers will follow to build the system.

Implementation:

- Transform the design into an operational system. Developers write the code to create the system based on the design.
- **Example:** The CRM system's features, like customer profiles and dashboards, are coded

Testing:

- Verify that the system meets the specified requirements and makes sure everything works as planned.
- The CRM system undergoes various testing procedures, such as unit testing, integration testing, and user acceptance testing, to ensure its functionality, performance, and security.

Deployment:

- All of the hard work concludes at the deployment phase, when the system is made accessible for real usage. After all the planning, designing, and building is finished, it's similar like launching a new store.
- This stage involves transferring the system from a testing or development environment to a production environment where actual users can access it.

Maintenance and support:

- Ensure the ongoing functionality and address any issues that arise.
- Regular updates, bug fixes, and user support for the CRM system to adapt to changing business requirements and address any emerging issues.

Models used for system design life cycle:

Waterfall model – A linear and sequential model where each phase must be completed before moving on to the next. It’s a straightforward approach but can be inflexible in the face of changing requirements.

Iterative model – Involves repeating cycles, with each iteration refining and improving the system based on feedback. It’s adaptable to changing requirements.

Prototyping model – Involves building a prototype (a preliminary version) of the system to gather feedback refine the design before building the final product.

Spiral model – Incorporates elements of both iterative and prototyping models. It involves cycles of planning, designing, constructing and evaluating.

Agile model – Emphasizes flexibility and collaboration, with frequent iterations and continuous feedback. It's well suited for projects where requirements may evolve.

Use cases:

- Developing new software applications.
- Enhancing existing software applications.
- Integrating different systems together.
- Replacing legacy systems.
- Developing custom solutions for specific business needs.